

Contar nós cuja soma dos filhos é maior que o próprio nó

Questão 5 · Questão em Grupo (2 pontos) · Árvore

Enunciado

Árvore: contar nós cuja soma dos filhos é maior que ele próprio.

Explicação passo a passo

Imagine que cada nó é um pai com até dois filhos. A pergunta é: "se eu somar os valores dos meus filhos, dá mais que o meu valor?". Se sim, esse pai entra na contagem.

- 1 Árvore vazia?** Não há ninguém para contar: devolve 0 (caso base).
- 2 O nó tem pelo menos um filho?** Se sim, soma os valores dos filhos que existem. Um filho que não existe vale 0.
- 3 A soma é maior que o valor do nó?** Então este nó conta 1.
- 4** Devolve o que este nó contou mais o que a subárvore esquerda contou mais o que a subárvore direita contou (duas chamadas recursivas).

Exemplo

- O **10** conta, porque $6 + 7 = 13$ é maior que 10.
- O **6** não conta, porque $2 + 3 = 5$ é menor que 6.
- O **7** conta, porque seu único filho, 9, é maior que 7.
- Os nós **2, 3 e 9** são folhas e não têm filhos para somar.

Observação: considere que folha (nó sem filhos) não entra na contagem, pois não existe "soma dos filhos" sem filhos. Se o professor considerar a soma vazia como 0, folhas com valor negativo entrariam. Com valores positivos o resultado é o mesmo.

Código completo

Arquivo: 05_arvore_soma_filhos.c. Para compilar: gcc -Wall -o programa 05_arvore_soma_filhos.c && ./programa

```
/* Árvore: contar nós cuja soma dos filhos é MAIOR que o próprio nó */
#include <stdio.h>
#include <stdlib.h>

typedef struct NoArvore {
    int valor;
    struct NoArvore* esq;
    struct NoArvore* dir;
} NoArvore;

NoArvore* criarNo(int v, NoArvore* e, NoArvore* d) {
    NoArvore* n = (NoArvore*) malloc(sizeof(NoArvore));
    if (!n) exit(1);
    n->valor = v; n->esq = e; n->dir = d;
    return n;
}
```

```

void liberarArvore(NoArvore* r) {
    if (!r) return;
    liberarArvore(r->esq); liberarArvore(r->dir); free(r);
}

int contarSomaFilhosMaior(NoArvore* raiz) {
    /* Caso base: árvore vazia não tem nó nenhum para contar */
    if (raiz == NULL) return 0;

    int conta = 0;

    /* Só faz sentido falar em "soma dos filhos" se existir pelo menos um filho */
    if (raiz->esq != NULL || raiz->dir != NULL) {
        int soma = 0;
        if (raiz->esq != NULL) soma += raiz->esq->valor; /* filho que não existe vale 0 */
        if (raiz->dir != NULL) soma += raiz->dir->valor;

        if (soma > raiz->valor) conta = 1; /* este nó entra na contagem */
    }

    /* Soma o que ele mesmo contou com o que os dois lados contarem */
    return conta + contarSomaFilhosMaior(raiz->esq) + contarSomaFilhosMaior(raiz->dir);
}

int main(void) {
    /*
        10
       / \
      6   7
     / \   \
    2  3   9
    Esperado: 2 nós (o 10, pois 6+7=13>10; o 7, pois 9>7). O 6 não: 2+3=5<6. */
    NoArvore* raiz = criarNo(10,
                             criarNo(6, criarNo(2, NULL, NULL), criarNo(3, NULL, NULL)),
                             criarNo(7, NULL, criarNo(9, NULL, NULL)));

    printf("Quantidade: %d\n", contarSomaFilhosMaior(raiz));
    printf("Arvore vazia: %d\n", contarSomaFilhosMaior(NULL));

    liberarArvore(raiz);
    return 0;
}

```

Teste executado

Árvore: 10 com filhos 6 e 7; o 6 com filhos 2 e 3; o 7 com filho direito 9. Resultado esperado: 2.

Quantidade: 2
Arvore vazia: 0